

Overview of Encoder decoder architecture

Subject: Encoder decoder architecture

1.

Tokenizers, which convert text into tokens. Embedding layer, which converts tokens and positions of the tokens into vector representations. Transformer layers, which carry out repeated transformations on the vector representations, extracting more and more linguistic information. These consist of alternating attention and feedforward layers. There are two major types of transformer layers: encoder layers and decoder layers, with further variants. Un-embedding layer, which converts the final vector representations back to a probability distribution over the tokens. The following description follows exactly the transformer as described in the original paper. There are variants, described in the following section. By convention, we write all vectors as row vectors. For example, pushing a vector through a linear layer means multiplying it by a weight matrix on the right, as xW .

2.

One set of (W^Q, W^K, W^V) matrices is called an attention head, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices, W^Q and W^K , which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix W^V , in combination with the part of the output projection matrix W^O , determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects. The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers. Concretely, let the multiple attention heads be indexed by i , then we have $\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i^Q, XW_i^K, XW_i^V))W^O$ where the matrix X is the concatenation of word embeddings, and the matrices W_i^Q, W_i^K, W_i^V are "projection matrices" owned by individual attention head i , and W^O is a final projection matrix owned by the whole multihead attention head. It is theoretically possible for each attention head to have a different head dimension d_{head} , but that is rarely the case in practice. As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions: $d_{\text{emb}} = 768$, $n_{\text{head}} = 12$, $d_{\text{head}} = 64$. Since $12 \times 64 = 768$

$12 \times 64 = 768$, its output projection matrix $W_O \in \mathbb{R}^{(12 \times 64) \times 768}$ is a square matrix.

3.

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$. Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

4.

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

5.

In words, it means that each token can pay attention to itself, and every token before it, but not any after it. A non-masked attention module can be thought of as a masked attention module where the mask has all entries zero. As an example of an uncommon use of mask matrix, the XLNet considers all masks of the form PM_{causal}^{-1} , where P is a random permutation matrix.

6.

where the softmax is applied over each of the rows of the matrix. The number of dimensions in a query vector is query size d_{query} and similarly for the key size d_{key} and value size d_{value} . The output dimension of an attention head is its head dimension d_{head} . The attention mechanism requires the following three equalities to hold: $d_{\text{seq, key}} = d_{\text{seq, value}}$, $d_{\text{query}} = d_{\text{key}}$, $d_{\text{value}} = d_{\text{head}}$ but is otherwise unconstrained. If the attention head is used in a self-attention fashion, then $X_{\text{query}} = X_{\text{key}} = X_{\text{value}}$. If the attention head is used in a cross-attention fashion, then usually $X_{\text{query}} \neq X_{\text{key}} = X_{\text{value}}$. It is theoretically possible for all three to be different, but that is rarely the case in practice.